

Reviewer Pack — Gröbner Basis Step Count vs. Extended Frege Proof Length for...

Entry 614c06db4715 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 06:23:40 UTC

Gröbner Basis Step Count vs. Extended Frege Proof Length for 3-SAT

Entry ID: 614c06db4715 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 06:23:40 UTC

1. Conjecture

Field A (mathematical branch): Computational Algebraic Geometry (Gröbner Basis Computation)

Field B (complexity object): Extended Frege Proof Length for 3-SAT Instances

Statement:

For a random 3-SAT instance with n variables, the number of S-polynomial reductions required to compute the Gröbner basis of its corresponding polynomial system is $\Theta(2^{\{n/2\}})$.

Rationale (proposer's reasoning):

Gröbner basis algorithms involve reduction steps similar to proof steps in Extended Frege, and the exponential complexity of 3-SAT suggests a similar exponential step count in Gröbner basis computation.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8e260346e5573ec2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_3sat_clause(clause):
        return len(clause) == 3 and all(x in {'x', 'y', 'z'} for x in clause)

    def generate_random_3sat(n, m):
        clauses = []
        variables = ['x', 'y', 'z']
        while len(clauses) < m:
            clause = [random.choice(variables) + random.choice(['', '"']) for _ in range(3)]
            if is_3sat_clause(clause):
                clauses.append(clause)
        return clauses

    def polynomial_from_clause(clause):
        x, y, z = 'x', 'y', 'z'
        clause_poly = 1
        for var in clause:
            if var[0] == 'x':
                coeff = -1 if var[1] == "" else 1
                clause_poly *= (coeff * x + int(var[1] != ""))
            elif var[0] == 'y':
                coeff = -1 if var[1] == "" else 1
                clause_poly *= (coeff * y + int(var[1] != ""))
            elif var[0] == 'z':
                coeff = -1 if var[1] == "" else 1
                clause_poly *= (coeff * z + int(var[1] != ""))
        return clause_poly

    def gcd(a, b):
        while b:
            a, b = b, a % b
        return a

    def lcm(a, b):
        return abs(a*b) // gcd(a, b)
```

```

def s_polynomial(f, g):
    m1, m2 = f[0], g[0]
    if m1 < m2:
        f, g = g, f
        m1, m2 = m2, m1
    lcm_monomial = 1
    for i in range(len(m1)):
        lcm_monomial *= lcm(m1[i], m2[i])
    s_poly = [0] * len(f)
    for i in range(len(f)):
        s_poly[i] = (lcm_monomial // m1[i]) * f[i]
        if m1[i] > m2[i]:
            s_poly[i] -= (lcm_monomial // m2[i]) * g[i]
    return s_poly

def reduce_polynomial(poly, monomials):
    for i in range(len(monomials)):
        while poly[0] >= monomials[i][0]:
            coeff = poly[1] / monomials[i][1]
            poly[1] -= coeff * monomials[i][2]
            poly[0] -= 1
    return poly

def buchberger_algorithm(poly_system):
    S_polynomials = []
    for i in range(len(poly_system)):
        for j in range(i + 1, len(poly_system)):
            s_poly = s_polynomial(poly_system[i], poly_system[j])
            if any(s_poly[k] != 0 for k in range(len(s_poly))):
                S_polynomials.append((s_poly[0], s_poly[1]))
    return S_polynomials

def count_spolynomials(poly_system):
    S_polynomials = buchberger_algorithm(poly_system)
    return len(S_polynomials)

n = random.randint(5, 40)
m = random.randint(n * 2, n * 3)
clauses = generate_random_3sat(n, m)
poly_system = [polynomial_from_clause(clause) for clause in clauses]

s_poly_count = count_spolynomials(poly_system)

expected_s_poly_count = math.ceil(2**(n/2))

return {
    "metric_name": "S-polynomial reductions",
    "metric_value": s_poly_count,
    "instances_tested": 1,
    "conjecture_holds": abs(s_poly_count - expected_s_poly_count) <= 0.1 * expected_s_poly_count,
    "counterexample": "" if abs(s_poly_count - expected_s_poly_count) <= 0.1 * expected_s_poly_count else ""
}

if __name__ == "__main__":
    import sys

```

```

seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_s_poly_count = sum(r["metric_value"] for r in results) / len(results)
std_dev = math.sqrt(sum((r["metric_value"] - mean_s_poly_count)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_s_poly_count} std={std_dev} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10aacbf.py", line 115, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10aacbf.py", line 95, in run_trial
    poly_system = [polynomial_from_clause(clause) for clause in clauses]
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10aacbf.py", line 44, in polynomial_from_clause
    coeff = -1 if var[1] == "" else 1
                ~~~^
IndexError: string index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to IndexError, preventing data collection. Pre-registered support condition cannot be evaluated without successful runs. | next: Fix the string indexing error in polynomial_from_clause() to enable proper test execution

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	33380
2	preregistration	ollama_remote	qwen3:8b	0	0	24074
3	novelty	ollama_remote	qwen3:8b	0	0	20673
4	novelty	ollama_remote	qwen3:8b	0	0	12898
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14656
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13595
7	judge	ollama_remote	qwen3:8b	0	0	17497

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 136774 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/614c06db4715.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/614c06db4715.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/614c06db4715.tar.gz` (if generated)